

**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Mención en Ingeniería del Software

Sistema de Visualización de Métricas para Equipos Ágiles con Scrum/Kanban

(Manual técnico, manual de usuario y manual de código)

Autor

Ángel Eduardo Bauste De Nicolais

Director(es)

María Luque Rodríguez
Francisco Gracia Ahufinger

Agosto, 2026



Declaración de originalidad del TFG

Ángel Eduardo Bauste De Nicolais con DNI nº 03527704X, estudiante del Grado en Ingeniería Informática,

DECLARA

Que es el autor del presente trabajo siendo fruto de su propio esfuerzo y no constituye una reproducción total o parcial de otro trabajo. Todas las ideas, datos o fragmentos de otras obras han sido claramente señaladas, citadas y referenciadas conforme a las normas académicas y éticas.

Sistema de Visualización de Métricas para Equipos Ágiles con Scrum/Kanban

Resumen

Este Trabajo Fin de Grado propone el desarrollo de un sistema web de visualización de métricas para equipos ágiles que trabajan con Scrum y Kanban. La herramienta permitirá centralizar, analizar y mostrar indicadores clave como Velocity, Cycle Time, Lead Time y WIP, con el objetivo de mejorar la toma de decisiones basada en datos y favorecer la mejora continua.

El proyecto surge ante la dificultad que presentan muchos equipos para obtener una visión unificada de sus métricas, debido a la dispersión de la información entre distintas herramientas y al elevado coste de algunas soluciones comerciales. Para dar respuesta a esta necesidad, se diseñará una plataforma personalizable que obtendrá sus datos principalmente de Jira e integrará dashboards intuitivos, de modo que los equipos puedan tomar decisiones informadas y evaluar con transparencia el rendimiento del proyecto.

Palabras clave: Métricas ágiles, Scrum, Kanban, dashboard, visualización de datos, mejora continua, Jira, analítica de software.

Agile Team Metrics Visualization System with Scrum/Kanban

Abstract

This Bachelor's Thesis proposes the development of a web-based metrics visualization system for agile teams working with Scrum and Kanban. The tool will enable the centralization, analysis, and display of key indicators such as Velocity, Cycle Time, Lead Time, and WIP, with the aim of improving data-driven decision-making and supporting continuous improvement.

The project addresses the common difficulty many teams face when trying to obtain a unified view of their metrics, due to data being scattered across different tools and the high cost of existing commercial solutions. To address this need, a customizable platform will be designed to obtain its data mainly from Jira and integrate intuitive dashboards, allowing teams to make informed decisions and assess project performance transparently.

Keywords: Agile metrics, Scrum, Kanban, dashboard, data visualization, continuous improvement, Jira, software analytics.

Agradecimientos

Me gustaría dedicar unas palabras de agradecimiento a las personas que han ayudado a la consecución de este proyecto. En primer lugar, ...

Índice general

Índice general	IX
Índice de figuras	XI
Índice de tablas	XIII
1 Introducción	1
1.1 Motivación	2
1.2 Objetivos generales	2
1.3 Estructura de la memoria	3
2 Definición del problema	5
2.1 Universidad: contexto de la labor docente	5
3 Requisitos	7
3.1 Funcionales	7
3.2 Casos de uso	7
3.3 Validación de casos de uso	7
Bibliografía	9
A Resultados detallados de la experimentación	11
Referencias	11
B Manual de usuario	15
Referencias	15
C Manual de código fuente	17
C.1 Contenidos del manual de código fuente	17
C.2 Código fuente de otros autores	18
C.3 ¿Qué va a mirar el tribunal en tú código?	18
C.4 Estilo del texto de los ficheros de código fuente	19
Referencias	21

Índice de figuras

2.1	Evolución de la población de 18 a 24 años, de los matriculados en Grado, 1 er y 2o ciclo y Máster y de la tasa neta de escolarización en educación universitaria [3]	6
A.1	Pantalla de acceso.	13
A.2	Pantalla de perfil de usuario.	13

Índice de tablas

3.1	CU30:Ordenar rutinas	7
3.2	Matriz de validación requisitos funcionales y casos de uso	8
A.1	Resultados de regresión logística en Buzzfeed	12
A.2	CU01: Registro en la aplicación	14

Capítulo 1

Introducción

En la industria actual del desarrollo de software, la gestión de proyectos se apoya con frecuencia en enfoques ágiles, implementados habitualmente mediante marcos de trabajo como Scrum y Kanban. En este contexto, las métricas se convierten en una herramienta fundamental para que los equipos puedan medir su rendimiento, evaluar la eficiencia de su flujo de trabajo y diagnosticar la salud general del proceso de desarrollo. Indicadores como la velocidad de entrega (Velocity), el tiempo de ciclo (Cycle Time), el tiempo de entrega (Lead Time) y el trabajo en curso (WIP) permiten tomar decisiones basadas en evidencia y avanzar hacia la mejora continua.

Sin embargo, a pesar de la importancia de estas métricas, muchos equipos encuentran dificultades para obtener una visión unificada y útil de las mismas. La información relativa al trabajo suele estar fragmentada en distintas herramientas (por ejemplo, Jira para la gestión de tareas, GitHub para el código o plataformas de despliegue), y las soluciones comerciales que prometen integrar y visualizar estos datos a menudo tienen costes elevados o están orientadas a grandes organizaciones. Esta falta de visibilidad lleva a que, en la práctica, muchas decisiones se tomen más por intuición que por datos objetivos, lo cual limita la capacidad de mejorar el proceso de forma sistemática.

Este Trabajo Fin de Grado surge como respuesta a esta problemática, con el objetivo de desarrollar un sistema web de visualización de métricas para equipos ágiles que trabajan con Scrum y Kanban. La herramienta permitirá centralizar, analizar y mostrar indicadores clave a partir de los datos almacenados en Jira, fuente principal del sistema, de modo que los equipos puedan contar con dashboards intuitivos y personalizados que faciliten la toma de decisiones basada en datos y la evaluación transparente del rendimiento del proyecto.

1.1. Motivación

La motivación principal de este proyecto es democratizar el acceso a la analítica ágil de calidad, ofreciendo una solución de bajo coste, personalizable y replicable, que pueda ser utilizada tanto por equipos profesionales como por estudiantes o grupos de trabajo con recursos limitados.

Actualmente, herramientas como Jira permiten gestionar el trabajo de forma eficiente, pero sus capacidades analíticas avanzadas suelen estar disponibles solo en planes de pago o requieren la integración con soluciones externas de terceros. Además, estas soluciones tienden a ofrecer dashboards genéricos, con poca flexibilidad para adaptarse a las necesidades específicas de cada equipo. Esto genera una barrera importante para equipos pequeños o medianos, que no pueden asumir el coste de licencias premium pero que, al mismo tiempo, necesitan visibilidad sobre sus métricas para mejorar su forma de trabajar.

El presente TFG busca cubrir ese hueco mediante una plataforma que:

- Centralice los datos procedentes de Jira, evitando la dispersión de la información.
- Calcule automáticamente métricas clave como Velocity, Cycle Time, Lead Time y WIP.
- Ofrezca dashboards intuitivos y personalizables, adaptados a las necesidades de cada equipo.
- Sea técnica y económicamente replicable, utilizando tecnologías modernas y de amplio uso en la industria.

De esta forma, se pretende devolver a los equipos el control sobre sus procesos de mejora continua, basándose en evidencia y no solo en intuición.

1.2. Objetivos generales

El objetivo principal de este Trabajo Fin de Grado es construir un dashboard web funcional que permita a los equipos ágiles visualizar, analizar y gestionar sus métricas clave de Scrum y Kanban mediante paneles intuitivos y personalizables.

Para alcanzar este objetivo general, se plantean los siguientes subobjetivos:

- Implementar un módulo de captura de datos que se integre con Jira Cloud API para automatizar la obtención de información relevante de proyectos, tableros, sprints e incidencias.

- Diseñar e implementar dashboards personalizables que permitan a cada equipo seleccionar y visualizar sus métricas ágiles más relevantes mediante una interfaz responsive.
- Desarrollar un sistema de alertas que notifique al equipo cuando se detecten anomalías o tendencias preocupantes en las métricas.
- Validar la usabilidad de la plataforma mediante pruebas con equipos ágiles o simulaciones, con el objetivo de alcanzar un nivel de satisfacción del usuario elevado.
- Implementar las medidas de seguridad y privacidad necesarias, incluyendo autenticación robusta a través de Jira/Atlassian y protección de información sensible.

1.3. Estructura de la memoria

Esta memoria se organiza en varios capítulos y anexos que recogen de forma progresiva el análisis, diseño, desarrollo y validación del sistema:

1. La presente introducción, donde se contextualiza el proyecto y se resumen su motivación y objetivos principales.
2. La definición del problema, dedicada a describir la situación actual de los equipos ágiles y las limitaciones que justifican el desarrollo del sistema.
3. El capítulo de requisitos, donde se especifican las funcionalidades, restricciones y necesidades de información que debe cubrir la solución.
4. Los capítulos técnicos de diseño, implementación y validación, que se completarán conforme avance el desarrollo del sistema.
5. Los anexos, que incluirán el manual de usuario, el manual de código y cualquier material complementario necesario para comprender, desplegar o mantener la aplicación.

Capítulo 2

Definición del problema

2.1. Universidad: contexto de la labor docente

Ejemplo de un texto marcado como literal de una fuente citada:

Las funciones básicas de la enseñanza universitaria, tal y como se concibe en la actualidad, fueron establecidas por José Ortega y Gasset en su trabajo «Misión de la Universidad» [1].

Ejemplo de estilo para citar un bloque de texto:

Las funciones de la Universidad propuestas por Ortega y Gasset se encuentran recogidas en la Ley Orgánica de Universidades (LOU) que establece:

Artículo 1. Funciones de la Universidad:

1. La Universidad realiza el servicio público de la educación superior, mediante la investigación, la docencia y el estudio.
2. Son funciones de la Universidad al servicio de la sociedad:
 - a) La creación, desarrollo, transmisión y crítica de la ciencia, de la técnica y de la cultura.
 - b) La preparación para el ejercicio de actividades profesionales que exijan la aplicación de conocimientos y métodos científicos y para la creación artística.
 - c) La difusión, la valorización y la transferencia del conocimiento al servicio de la cultura, de la calidad de vida, y del desarrollo económico.
 - d) La difusión del conocimiento y la cultura a través de la extensión universitaria y la formación a lo largo de toda la vida.

(Ley Orgánica de Universidades [2])

Ejemplo de figura mencionada en el texto: Según los datos del Ministerio de Educación, Cultura y Deporte publicados en su memoria del curso 2015-2016 [3], que es la más actual disponible en la fecha de realización de este trabajo, los datos del número de alumnos apuntan a una estabilización en los últimos años. Sin embargo, si se observan los datos de forma relativa a la población siguen mostrando un crecimiento importante. En la figura 2.1, se pueden observar estas tendencias.

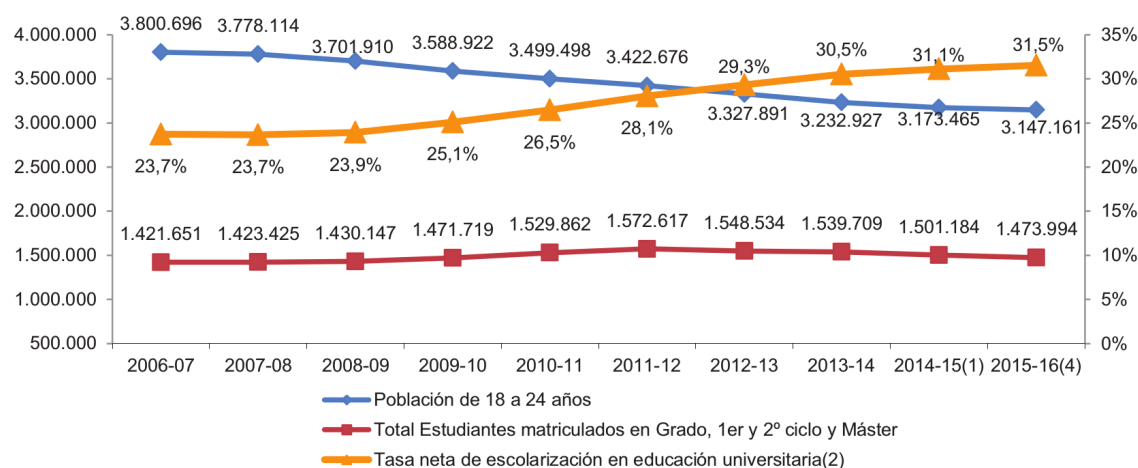


Figura 2.1: Evolución de la población de 18 a 24 años, de los matriculados en Grado, 1 er y 2o ciclo y Máster y de la tasa neta de escolarización en educación universitaria [3]

Capítulo 3

Requisitos

3.1. Funcionales

...

3.2. Casos de uso

Ejemplo de como se podría formatear una tabla de caso de uso

Caso de uso	1 Ordenar Rutinas
Actores	Administrador y Cliente
Descripción	Se selecciona el filtro y se ordena las rutinas.
Precondición	El usuario está logueado en el sistema.
Flujo principal	1. El Caso de Uso comienza cuando el usuario selecciona la opción de ordenar rutinas. 2. El usuario selecciona el filtro para ordenar. 3. El sistema pide el dato para ordenar. 4. El usuario introduce el dato. 5. El sistema devuelve la lista ordenada.
Postcondición	Lista de rutinas ordenadas por un filtro.
Flujo Alternativo	Cancelar: el flujo alternativo comienza en cualquier punto del flujo principal. El usuario pulsa cancelar y la operación queda cancelada sin realizar ningun cambio.

Tabla 3.1: CU30:Ordenar rutinas

3.3. Validación de casos de uso

Ejemplo de una matriz de validacion de casos de uso.

A continuación se lleva a cabo la validación de los casos de uso mediante una matriz de trazabilidad que relaciona los casos de uso con los requisitos funcionales, de manera que es posible comprobar que todos los casos de uso están cubiertos por dichos requisitos y viceversa.

	Casos de uso																												
	1	2	3	4	5	6	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
RF-1				X	X																								
RF-2						X																							
RF-3							X																						
RF-4								X																					
RF-5									X																				
RF-6	X																												
RF-7		X																											
RF-8		X							X																				
RF-9										X																			
RF-10												X																	
RF-11												X	X																
RF-12											X																		
RF-13														X															
RF-14																						X							
RF-15																						X							
RF-16															X														
RF-17																							X						
RF-18																								X					
RF-19																X										X			
RF-20																	X												
RF-21																			X										
RF-22																		X											
RF-23			X																										
RF-24	X	X	X	X	X	X	X	X	X						X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
RF-25										X																X			
RF-26																					X								

Tabla 3.2: Matriz de validación requisitos funcionales y casos de uso

Bibliografía

- [1] J. Ortega y Gasset. «Obras de J. Ortega y Gasset». En: ed. por Paulino Garagorri. Alianza Editorial, 1982. Cap. Misión de la Universidad, págs. 313-353.
- [2] «Ley Orgánica 6/2001, de 21 de diciembre, de Universidades (LOU)». En: *BOE* 307 (24 de diciembre de 2001). Última actualización 28 de junio de 2017, págs. 49400-49425. URL: <https://www.boe.es/buscar/act.php?id=BOE-A-2001-24515>.
- [3] Cultura y Deporte Ministerio de Educación. *Datos y Cifras del Sistema Universitario Español 2015-2016*. URL: <https://www.mecd.gob.es/servicios-al-ciudadano-mecd/estadisticas/educacion/universitaria/datos-cifras.html> (visitado 18-11-2017).

Apéndice A

Resultados detallados de la experimentación

En este apéndice se incluyen una serie de tablas que se ha considerado que, incluidas dentro del capítulo de experimentación, dificultaban su lectura y la comprensión de la estructura experimental.

Estas tablas describen...

- BuzzFeed [1]: tablas A.1 a A.1.
- Politifact: ...

Ejemplo de estilo para poner dos figuras en paralelo:

Ejemplo para casos de uso:

Referencias

- [1] BuzzFeedNews. *Fact-Checking Facebook Politics Pages*. 2016. URL: <https://github.com/BuzzFeedNews/2016-10-facebook-fact-check/> (visitado 09-06-2020).

Tabla A.1: Resultados de regresión logística en Buzzfeed

Fichero	Balanceo	% Clase mayoritaria	Acc. (%) entrena- miento	Acc. (%) test
fase_1_y _2_data.cs v	Sin balanceo	84	85.7 ± 0.6	84.0 ± 2.7
fase_1_y _2_data.cs v	Upsample	50 en train y 84 en test	79.8 ± 1.8	72.7 ± 4
fase_1_y _2_data.cs v	Downsample	50 en train y 84 en test	79.0 ± 3.3	71.1 ± 3.5
fase3_da ta_1_Gra ms.csv	Sin balanceo	84	98.5 ± 1.8	86.4 ± 4.1
fase3_da ta_1_Gra ms.csv	Upsample	50 en train y 84 en test	99.5 ± 0.1	86.3 ± 3.4
fase3_data _1_Grams .csv	Downsample	50 en train y 84 en test	95.4 ± 1.03	74.7 ± 6.4
fase3_data _2_Grams .csv	Sin balanceo	84	99.2 ± 0.2	85.0 ± 5.2
fase3_data _2_Grams .csv	Upsample	50 en train y 84 en test	99.5 ± 0.001	84.7 ± 4.3
fase3_da ta_2_Gra ms.csv	Downsample	50 en train y 84 en test	94.4 ± 7.3	77.6 ± 2.5
fase3_data _3_Grams .csv	Sin balanceo	84	88.7 ± 7.2	84.3 ± 1.7
fase3_data _3_Grams .csv	Upsample	50 en train y 84 en test	99.2 ± 0.4	81.8 ± 3.1
fase3_data _3_Grams .csv	Downsample	50 en train y 84 en test	93.3 ± 8.3	77.6 ± 5.8

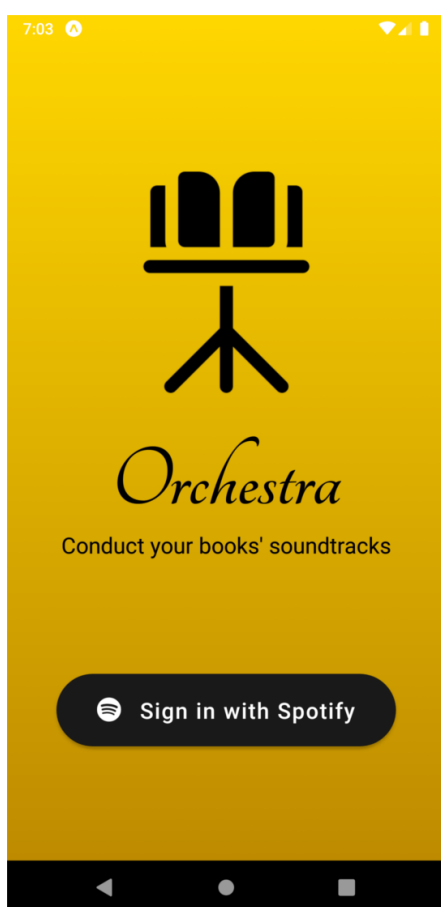


Figura A.1: Pantalla de acceso.

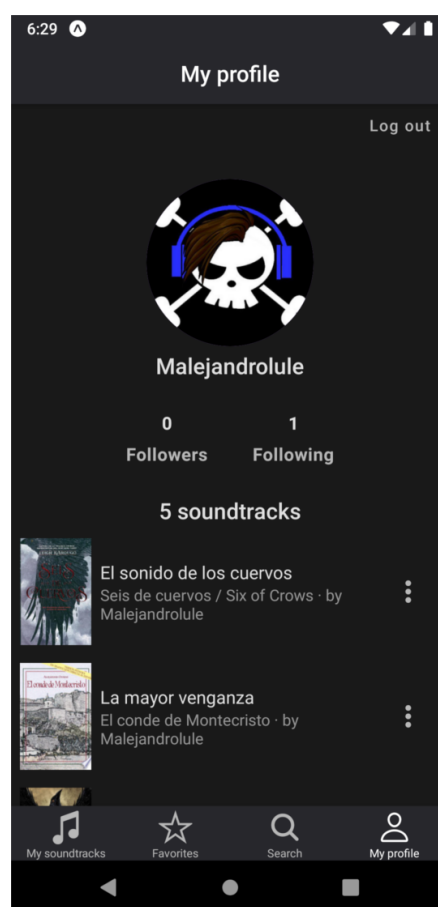


Figura A.2: Pantalla de perfil de usuario.

Caso de Uso	Nombre del caso de uso
Actores	Usuario no registrado.
Descripción	El objetivo del caso de uso es que cualquier usuario pueda registrarse para hacer uso de la aplicación.
Precondición	El usuario no debe estar previamente registrado en la aplicación.
Flujo principal	1. El Caso de Uso comienza cuando el usuario selecciona la opción de registro.
	2. La Aplicación solicita al usuario un email y una contraseña.
	3. El usuario introduce el email.
	4. El usuario introduce la contraseña.
	5. La aplicación valida que el email y la contraseña es correcto.
	6. Mientras la contraseña o email no sean correctos.
	a) La aplicación solicita al usuario un email.
	b) El usuario introduce el email.
Flujo principal	c) La aplicación solicita al usuario una contraseña.
	d) El usuario introduce la contraseña.
	e) La aplicación valida los datos son correctos.
	7. El usuario selecciona la opción de finalizar registro
	8. La aplicación informa al usuario que se ha registrado correctamente.
Postcondición	El usuario se ha registrado aplicación.
Flujo Alternativo	Cancelar: el usuario cancela el registro y vuelve a la página de Inicio.

Tabla A.2: CU01: Registro en la aplicación

Apéndice B

Manual de usuario

Aquí el manual de usuario. También podría ponerse en un documento a parte. Hay quien prefiere que sean documentos separados porque en la realidad deben serlo ya que el destinatario es distinto. Hay quien prefiere que estén como anexos porque así en un documento único está todo.

Apéndice C

Manual de código fuente

C.1. Contenidos del manual de código fuente

- Introducción del documento, explicando: a que proyecto corresponde, que lenguajes de programación, entornos, o librerías se han utilizado, y todo aquello que tenga relación con como se ha creado y organizado el código fuente. (obligatorio)
- Instrucciones de compilación, empaquetado, actualización... (si proceden)
- Instrucciones de instalación y desinstalación (si procede aquí en vez de en el manual de usuario)
- Descripción de la organización del código en archivos y como se corresponde con la organización modular explicada en la memoria técnica. (obligatorio)
- Sí ayuda a la comprensión de la organización del código y se considera útil, documentación generada por sistemas automáticos de documentación, o breve explicación y enlace a la web (ej.) donde se puede encontrar esta documentación. (si procede)
- Todos los ficheros con el código fuente de la aplicación (ej. código fuente .py, código fuente .c, cabeceras .h, Makefile, configure, ficheros de creación de bases de datos .sql, páginas web .html, código fuente .php, ficheros de carga de datos básicos, scripts del sistema operativo .sh....) (obligatorio, puede ser en repositorio accesible)

En vez de preparar todo el código con estilo de impresión, es adecuado incluir el código en un repositorio donde el tribunal pueda consultarlo indicandolo en este manual. En ese caso, ya no es necesario poner el código en este manual pero sí las explicaciones de su estructura. Ejemplo:

... todo el código aquí descrito se puede encontrar en el siguiente repositorio: <https://gitlab.com/angelpavon97/fake-news-detector/>

C.2. Código fuente de otros autores

No se debe incluir código fuente que no haya sido desarrollado por el autor para el proyecto.

- Si hay ficheros completos tomados de otras fuentes que son necesarios para el proyecto, estos deberán ser referenciados y explicar como se integran en el proyecto.
- Si se han realizado modificaciones a código de otras fuentes, cuando se incluyan estos ficheros modificados, se deberá explicar claramente su origen y el alcance de las modificaciones realizadas. Todo esto debe quedar muy claro, de forma que no dé lugar a duda sobre la intención del autor al incluirlo en el manual.
- Nota: en cualquier caso, el autor debe recordar que debe cumplir con toda la legislación sobre derechos de propiedad intelectual.

C.3. ¿Qué va a mirar el tribunal en tú código?

Puede ver que el código esté desarrollado de forma adecuada. Por ejemplo, en una ocasión vi una secuencia de una centena de instrucciones `if` encadenadas que claramente deberían haber sido sustituidas por un vector de datos y el acceso al mismo. Otra cosa que se puede es si hay valores insertados en el código que deberían ser configuraciones (*hard-coded*). Si has hecho un código con un mínimo de cuidado, no es algo que deba preocuparte.

También puede mirar que el estilo de código sea consistente (igual a lo largo de todo el código) y adecuado (nombres de variables descriptivos, no ofuscado...). Además, que se incluyen comentarios adecuados, que ayuden entender rápidamente el código y trabajar con él, a la vez que se siguen las reglas de Ottinger:

1. Comments are for things that cannot be expressed in code.
2. Comments which restate code must be deleted.
3. If the comment says what the code could say, then the code must change to make the comment redundant.

C.4. Estilo del texto de los ficheros de código fuente

Es importante que para incluir el código fuente se sigan algunas normas básicas. Se debe recordar siempre que la finalidad es facilitar el estudio y comprensión del código. Por tanto, éste deberá aparecer de la forma más parecida posible a como aparece en el editor y es mejor tener la mayor cantidad posible de código en cada página para reducir al máximo el tener que andar pasando páginas hacia atrás y hacia delante. No debe en ningún caso usar un estilo con el objeto de engordar el código y que aparente mayor volumen de trabajo (en muchos casos esto consigue justo lo contrario).

El estilo para el código fuente debería:

- Tener espaciado entre líneas y párrafos a cero.
- Fuentes de tamaño fijo o monoespacio (todos los caracteres ocupan lo mismo). Esto hace que los indentados para indicar los bloques de código se vean bien y que todo el código se alinee igual que en el editor.
- Un tamaño de fuente pequeño, para evitar, o al menos reducir, que las líneas no quepan y se pasen a la siguiente. Como el código no va a ser leído de forma continuada, no importa que la letra tenga un tamaño más reducido que en el resto de la documentación, pero debe ser un tamaño legible.

También es posible jugar con el formato (reducir márgenes, ponerlo a doble columna) para que ocupe menos páginas. A continuación se mostrarán ejemplos de código estilo.

Ejemplo de cita al lenguaje de programación:

... se ha realizado con el lenguaje de programación Python [1] ...

Ejemplo de un listado de directorio:

```
BaseDeDatosEjemplo
├── data
│   ├── True
│   │   ├── 1.txt
│   │   ├── 2.txt
│   │   └── 3.txt
│   └── False
│       ├── 1.txt
│       ├── 2.txt
│       └── 3.txt
├── BaseDeDatosEjemplo_1grams.csv
├── BaseDeDatosEjemplo_2grams.csv
├── BaseDeDatosEjemplo_3grams.csv
├── BaseDeDatosEjemplo_reduced_1grams.csv
├── BaseDeDatosEjemplo_reduced_2grams.csv
├── BaseDeDatosEjemplo_reduced_3grams.csv
├── fase1_data.csv
├── fase2_data.csv
├── fase3_data_1_Grams.csv
├── fase3_data_2_Grams.csv
├── fase3_data_3_Grams.csv
└── fase_1_y_2_data.csv
```

Ejemplo de listado código fuente Python:

```

1 import csv
2 import requests
3 import urllib.request
4 from bs4 import BeautifulSoup
5
6 def scraping_abc(url):
7     """
8     Comentario de la función
9     """
10    try:
11        page = urllib.request.urlopen(url)
12    except:
13        print("An error occurred.")
14        return 'error', 'error'
15
16    soup = BeautifulSoup(page, 'html.parser')
17
18    # Buscamos todos los parrafos (contenido)
19    content_lis = soup.find_all('p')
20
21    # Buscamos el titulo
22    header = soup.find('header', attrs={'class': 'Article__Header'})
23    title = header.find('h1').getText()
24
25    # Imprimimos
26    print('Title: ' + title)
27    content = ''
28    for p in content_lis:
29        content = content + p.getText()
30    print(content)
31
32    return title, content

```

Y un HTML:

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <meta http-equiv="X-UA-Compatible" content="ie=edge">
8     <!-- Bootstrap 4 -->
9     <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/
10         css/bootstrap.min.css"
11         integrity="sha384-Vkoo8x4CGsO3+Hhvx8T/
12         Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh" crossorigin="anonymous">
13
14     <!-- Custom css -->
15     <link rel="stylesheet" href="{url_for('static', filename='css/main.css')}">
16
17     <script src="{url_for('static', filename='js/app.js')}"></script>
18
19     <title>Fake News Detector</title>
20 </head>
21
22 <body>
23     <nav class="navbar navbar-expand-lg navbar-light bg-light">
24         <a class="navbar-brand" href="/">Fake News Detector</a>
25         <button class="navbar-toggler" type="button" data-toggle="collapse"
26             data-target="#navbarText"
27             aria-controls="navbarText" aria-expanded="false" aria-label="Toggle
28                 navigation">
29             <span class="navbar-toggler-icon"></span>
30         </button>
31         <div class="collapse navbar-collapse" id="navbarText">

```

```
28         <ul class="navbar-nav ml-auto">
29             <li id="nav-home" class="nav-item">
30                 <a class="nav-link" href="/">Home</a>
31             </li>
32             <li id="nav-project" class="nav-item">
33                 <a class="nav-link" href="{{url_for('project')}}">The Project</a>
34             </li>
35             <li id="nav-classifier" class="nav-item">
36                 <a class="nav-link" href="{{url_for('classifier')}}">Classifier
37                     </a>
38             </li>
39         </ul>
40     </div>
41 </nav>
42
43     {% block content %}
44     {% endblock %}
45 </body>
46
47 </html>
```

Referencias

- [1] Python Software Foundation. *Python (Versión 3.7) [Software]*. 2018. URL: <http://www.python.org> (visitado 02-07-2020).

